

Smart Light Bulb Security Analysis

Aayush Dhanotiya
Heinz College
CMU
Pittsburgh PA USA
adhanoti@andrew.cmu.edu

Supriya Nittoor
Heinz College
CMU
Pittsburgh PA USA
srnittoo@andrew.cmu.edu

Nicholas Park
INI
CMU
Sunnyvale CA USA
hyunsoo2@andrew.cmu.edu

Wendy Chen
ECE
CMU
San Jose CA USA
weichung@andrew.cmu.edu

Sterling McTee
Heinz College
CMU
Pittsburgh PA USA
smctee@andrew.cmu.edu

ABSTRACT

This research paper conducts a comprehensive security analysis of smart light bulbs, focusing on brands like Philips, Merkury, and Cync. The study employs a variety of testing methods, including static and dynamic analysis, to uncover vulnerabilities in these smart devices. It evaluates the potential impact of these vulnerabilities, comparing the security posture of bulbs across different price ranges. The paper also offers recommendations for enhancing device security, contributing insights into the current state of IoT security and providing valuable guidelines for consumers and manufacturers. The findings highlight the importance of robust security measures in smart home devices, emphasizing the need for continuous improvement in IoT security standards.

1 Introduction

The advent of smart home devices has ushered in a new era of convenience and automation, significantly transforming the way we interact with our living spaces. Among these innovations, smart bulbs have emerged as a cornerstone of modern home automation systems, offering users unprecedented control over illumination settings through mobile applications and voice commands. This surge in adoption has undeniably enriched our daily lives, making smart bulbs an integral component of contemporary living.

However, as the prevalence of smart home devices continues to escalate, so does the potential for cybersecurity vulnerabilities. The convenience they provide is accompanied by an increased risk of unauthorized access, data breaches, and exploitation by malicious actors. In this context, it becomes imperative to scrutinize the security measures implemented by these devices to ensure consumer safety and contribute to the ongoing

enhancement of Internet of Things (IoT) security.

2 Motivation

The proliferation of smart bulbs from various brands, including industry leaders like Philips, Merkury, and CYNK, necessitates a thorough examination of their security protocols. Recognizing the critical need for a safer digital ecosystem, this project is motivated by a commitment to comprehensively assess the security posture of these smart bulbs and their associated mobile applications.

The objective is to employ rigorous testing methodologies, including penetration testing, reverse engineering, and vulnerability scanning, to identify and analyze potential weaknesses within the smart bulbs' security infrastructure. This ethical exploration seeks to emulate the actions of potential malicious actors, ensuring that any vulnerabilities discovered are addressed proactively to mitigate risks.

The outcomes of these assessments are intended to serve a threefold purpose. Firstly, they aim to empower consumers with insights into the security of smart bulbs, enabling informed decisions about the devices they integrate into their homes. Secondly, the results will provide valuable feedback to manufacturers, fostering an iterative improvement process in their products' security features. Lastly, the findings can contribute to the formulation of guidelines and standards by governing bodies, shaping the future landscape of IoT security.

In essence, this initiative is not only timely but also indispensable for creating a safer and more secure digital environment. By methodically probing and analyzing the

security measures of smart bulbs from different brands, we seek to generate data that contributes to a nuanced understanding of the current state of smart bulb security, ultimately promoting the advancement of IoT security on a broader scale.

3 Goals

3.1 Identify Vulnerabilities

Conduct a systematic exploration of security measures implemented in smart light bulbs and their corresponding mobile applications from renowned brands, including Cync, Merkury, and Philips. Employ industry-standard methodologies such as penetration testing, reverse engineering, and vulnerability scanning to meticulously uncover and document security vulnerabilities.

3.2 Assess Impact

Evaluate the potential ramifications of identified vulnerabilities, considering both software vulnerabilities that may lead to unauthorized access or data leakage and physical vulnerabilities that could result in damage to the devices. Provide a comprehensive analysis of the impact of these vulnerabilities on the overall security and functionality of the smart light bulbs.

3.3 Compare Security Posture

Analyze and compare the security and privacy features of smart light bulbs across diverse price ranges. Examine whether there is a discernible correlation between the cost of the devices and the efficacy of their security measures. This comparative analysis aims to offer insights into the relationship between financial investment and the level of security provided by smart light bulb products.

3.4 Recommendations

Based on the findings from vulnerability assessments and comparative analysis, formulate detailed recommendations for mitigating the identified vulnerabilities and enhancing the overall security and privacy of smart light bulbs. These recommendations should encompass both technical and procedural measures, providing actionable insights for manufacturers and users alike to fortify the security posture of these devices.

4 Models and Assumptions

4.1 Assumptions about Smart Bulb Security

For this study, we operated under the primary assumption that smart bulbs, as increasingly common components of smart home systems, are inherently vulnerable to security threats. This assumption is grounded in the understanding that any network-connected device potentially offers entry points for unauthorized access or malicious activities. In the rush to market, we assumed developers might prioritize features over security, especially in the Internet of Things (IoT) sector. We hypothesized that despite their seemingly innocuous nature, smart bulbs could be exploited for data breaches, unauthorized control, or as gateways to larger network intrusions. This assumption directed our focus towards identifying and understanding potential vulnerabilities in smart bulb systems.

4.2 System Operation Models

We looked at smart bulbs as part of a larger ecosystem, involving their interaction with apps, users, and networks. It's like examining how a new appliance fits into the daily routine of a household. Our assumption was that in their quest to simplify life, these bulbs, with features like remote control and automation system integration, might also introduce new vulnerabilities. For example, the way a smart bulb connects to a home network might inadvertently create openings for security breaches.

4.3 Testing Methodologies as Models

Our testing methodologies including both static and dynamic analysis (including tools like MOBSF, Ghidra, APKlab, BurpSuite, WireShark, JADX, Frida) were selected based on the assumption that they are effective in uncovering a broad spectrum of vulnerabilities. We presumed that these methods would provide comprehensive insights into the software architecture, network interactions, and potential security loopholes in smart bulbs.

4.4 Environmental Assumptions

We assumed a typical home network environment for smart bulbs, with regular user behavior and typical network setups. It's like considering how a new electronic gadget would fit into an average household. This included assumptions about the types of devices and networks they would commonly interact with, as well as potential external threats, such as network eavesdropping or unauthorized access attempts.

4.5 Assumptions Based on Brand and Price Range

We initially assumed in our study that the price and brand of a smart bulb are indicative of its security level. We hypothesized that higher-priced bulbs, often from more

established brands, would exhibit stronger security features compared to lower-priced alternatives. This assumption was based on the notion that higher investment in product development, including security, is reflected in the product's retail price. However, our findings challenged this assumption, showing that while security tends to improve with price, there's a plateau where most brands offer adequate security features, regardless of the cost.

4.6 Limitations and Constraints

We understand our approach has its blind spots. For example, the link between price and security isn't a hard and fast rule. There might be biases in how we used our tools or gaps in our understanding of how these bulbs are used day-to-day.

4.7 Impact of Assumptions on Conclusions

Our assumptions were like guideposts, shaping our investigation and interpretation of data. For example, we realized that the correlation between price and security isn't as straightforward as we thought. This highlights the need for continuous evaluation and adaptation of our security models. Like updating a home security system to keep up with new threats, our models and assumptions require regular review and adaptation.

5 Testing Methods (Approach)

Since most of our members were working on different smart bulbs, all three have different approaches. Each sub-team was working on each bulb as well as using different additional tools to help their testing.

5.1 Merkury

5.1.1 Static Analysis Tools – DeGuard and MobSF

For an exhaustive assessment of the Merkury application's security, a combined static analysis approach utilizing both *DeGuard* and *MobSF* (Mobile Security Framework) was implemented.

The *DeGuard* tool played a pivotal role in deobfuscating the application, revealing the reversed Java source code. This process resulted in the extraction of an output.apk containing the deobfuscated codebase, which serves as a fundamental resource for subsequent analyses using tools such as *apktool*.

While *MobSF* is traditionally associated with dynamic analysis, its capabilities were leveraged for enhanced static

analysis as well. The tool provided insights into both static characteristics, such as code structure and permission usage, and dynamic behaviors, including runtime API calls. This dual functionality enriches the overall security assessment.

The synergy between static analysis through *DeGuard* and the expanded static and dynamic capabilities of *MobSF* establishes a robust foundation for subsequent phases, including penetration testing and a comprehensive evaluation of the Merkury application's security posture.

5.1.2 Dynamic Analysis Preparation – Jailbreaking

To facilitate dynamic analysis, the testing approach included the process of device jailbreaking. The objective was to elevate the device privileges and gain superuser access, enabling a more in-depth examination of the Merkury application's runtime behaviors and interactions.

In this endeavor, a Pixel device was successfully rooted using Magisk, a reputable and widely-used tool for rooting Android devices. The rooting process was meticulously executed, and the device was subsequently accessed with superuser privileges. This elevated access allows for a more comprehensive exploration of the application's activities and interactions with the underlying operating system.

The decision to root the Pixel device was guided by the need for an unrestricted view of the application's behavior, which is often constrained within the confines of a non-rooted environment. By obtaining superuser access, the dynamic analysis process gains the capability to scrutinize the application's interactions with the device at a deeper level, enhancing the overall assessment.

The rooting process was undertaken in accordance with established best practices. This ensures a reliable and secure foundation for subsequent network analysis using tools such as *BurpSuite*.

5.1.3 Network Analysis – BurpSuite

With the Pixel device successfully rooted and granted superuser access, the dynamic analysis phase extends to network analysis utilizing *BurpSuite*. *BurpSuite* serves as a powerful tool for intercepting, analyzing, and manipulating network traffic between the Merkury application and external servers.

This network analysis aims to unveil potential vulnerabilities related to data transmission, security protocols, and interactions with external services. By intercepting and inspecting the communication between the Mercury application and remote servers, it becomes possible to identify and assess any security risks or vulnerabilities associated with network activities.

The comprehensive dynamic analysis, coupled with the rooted device's elevated privileges, forms a robust approach to delve into the Mercury application's runtime behaviors and interactions, thereby contributing valuable insights to the overall security assessment.

5.1.4 Network Analysis – WireShark

A controlled wireless network broadcast by an external wireless AP was used for traffic analysis of Mercury traffic. The mobile device and smart bulbs were connected to this network, which was bridged with an Internet-connected WLAN through a Windows PC.

This method of analysis allowed detailed examination of mobile application traffic and of bulb traffic, since the bulb independently connects to the internet.

5.2 Cync by GE

5.2.1 Static Analysis Tools – Deobfuscation Tools (APKlab, Ghidra, MOBSF)

For Cync smart bulb application three tools were mainly used to deobfuscate and analyze. This process involved the utilization of APKlab, MobSF, and Ghidra, each contributing uniquely to the analysis.

Initially, APKlab was employed as the primary deobfuscation tool. It played a crucial role in unraveling the complexities of the application's code. APKlab effectively demystified the code, providing clearer insights into its structure and potential vulnerabilities. This initial phase laid the groundwork for deeper analysis.

For the second phase, MobSF (Mobile Security Framework) was brought into the fold. While it is often recognized for its dynamic analysis capabilities, in this context, MobSF was leveraged to augment the static analysis. It provided a more nuanced view of the application's architecture, permission usage, and potential security loopholes. This dual application of MobSF enriched the understanding of both the static and dynamic aspects of the Cync application.

The third phase involved the utilization of Ghidra, a sophisticated tool known for its prowess in software reverse engineering. Ghidra facilitated a deeper dive into the application's compiled code, offering valuable insights into its inner workings and potential areas of concern from a security standpoint.

5.2.1 Dynamic Analysis Tools – WireShark

For the dynamic analysis of the Cync application, the focus shifted to monitoring and analyzing the network traffic and runtime behavior of the application. This was achieved using Wireshark and a series of custom scripts.

5.3 Philips Hue

5.3.1 Static Analysis Tools – Deobfuscation Tools (apkdeguard, JADx, dex2jar, JD-GUI)

The first static analysis was to get hold of the code behind the application for further analysis. The initial attempt was to transfer the classes files we had found in the rooted Google Pixel device (where the application was installed) and then use the tools “dex2jar” as well as “JD-GUI” to uncover some of the code. However, as one can see in the figure below, the two tools still resulted in very heavily obfuscated code.

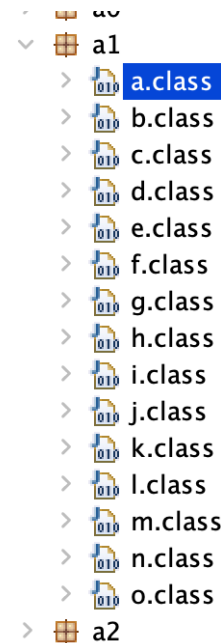


Figure 1: dex2jar and JD-GUI still returns very heavily obfuscated code

Thus, the second attempt was to use the tools “apkdeguard” and “jadx” which resulted in a better yet still obfuscated outcome. Results will be elaborated in section 6.3.1.

5.3.2 Static Analysis Tools – MobSF

The next tool that was used was MobSF, which is an automated static analysis tool to see if it could do a better job at not only deobfuscating the application code but also finding interesting vulnerabilities that would be otherwise difficult during a manual analysis.

In detail, some of the planned testing methods included the following. First, based on the permissions that were revealed by MobSF, cross check those permissions with the privacy and security policy manual of the application to see if the app is abusing any permissions. Second, if any potential vulnerabilities are spotted by MobSF, see if we can move them to the dynamic analysis portion and test them on the application.

5.3.2 Dynamic Analysis Tools – Frida

For the dynamic analysis, one ambitious idea that was discussed with the team was to hopefully modify the code to test the brightness level of the bulb and thus see if there were any negative consequences.

Therefore, the proposed testing method that was set forth was to use Frida to inject a personalized script that would overwrite or modify the brightness of the bulb that was currently running.

In order to execute this successfully, the testing method would include attaching the Google Pixel device to the Macbook Pro. The Pixel device would be interacting directly with the smart bulb while the Frida software in the Macbook Pro (that is currently linked with the Pixel device) would inject the script from the Macbook into the Philips Hue application.

6 Summary of Results

As our members primarily engage in analyzing different smart bulbs, each of them adopted distinct approaches.

6.1 Merkury

6.1.1 Static Analysis MOBSF

Permission Analysis

During static analysis, meticulous attention was directed toward scrutinizing the permissions utilized by the Merkury application. Notable concerns emerged with permissions of the following:

```
android.permission.ACCESS_BACKGROUND_LOCATION
android.permission.BLUETOOTH_SCAN
android.permission.CHANGE_WIFI_STATE
```

These permissions prompt focused investigation into potential privacy implications and security considerations. Significant concerns arise, especially with *ACCESS_BACKGROUND_LOCATION* indicating potential background location access.

A more in-depth examination of the deobfuscated code is warranted to understand the specific implementation of identified permissions, emphasizing potential vulnerabilities or privacy issues.

6.1.2 Static Analysis – Manifest Analysis

During the scrutiny of the Merkury application's *AndroidManifest.xml*, a noteworthy security concern emerged. An Activity-Alias was identified as being shared with other applications on the device, with the attribute *[android:exported=true]*. This configuration renders the service accessible to any other application on the device, potentially leading to security risks associated with unauthorized access.

The shared Activity-Alias instances include:

```
com.linecorp.linesdk.auth.internal.LineAuthenticationCallba
ckActivity
com.thingclips.smart.hometab.activity.shortcut
com.thingclips.smart.hometab.activity.speech
com.thingclips.smart.map.generalmap.MapModuleEntrance
com.thingclips.social.amazon.activity.TripleAlexaAccountLin
kActivity
```

This finding raises concerns about the potential for unintended interactions between the Merkury application and other apps on the device, emphasizing the need for a more restrictive configuration to mitigate security risks.

6.1.3 Static Analysis – Hardcoded Secrets

The analysis uncovered hardcoded secrets within the Merkury application's codebase, posing a significant security risk. Notable hardcoded secrets include:

```
"google_api_key":  
"AlzaSyBkaaDe8Hxnu0o659zFg-jTc9ahgqATH34"
```

```
"google_maps_key":  
"AlzaSyDK8oXjsnDQ-fUqnFt36i1XB_EJR6jIRW0"
```

The inclusion of hardcoded secrets introduces vulnerabilities, as these keys are exposed within the application's code, making them susceptible to unauthorized access or misuse. A best practice would involve utilizing secure storage mechanisms or dynamic retrieval to enhance the protection of sensitive information.

6.1.4 Static Analysis – Code Analysis

I. Encryption Vulnerability

The code analysis unveiled a vulnerability in the encryption mechanism employed by the Merkury application. Specifically, the app utilizes the encryption mode CBC with PKCS5/PKCS7 padding. This configuration is known to be vulnerable to padding oracle attacks, presenting a potential avenue for exploitation by malicious actors.

Given the critical role encryption plays in securing sensitive data, it is imperative to address this vulnerability promptly to enhance the overall resilience of the application against cryptographic attacks.

II. Insecure WebView Implementation

An additional concern surfaced during the analysis of the code related to the implementation of WebView. The application's WebView disregards SSL certificate errors and accepts any SSL certificate. This renders the application vulnerable to Man-in-the-Middle (MITM) attacks, where attackers can intercept and manipulate communication between the application and external servers.

To mitigate this vulnerability, it is recommended to implement proper SSL certificate validation mechanisms within the WebView, ensuring the secure and authenticated exchange of data.

These findings collectively underscore the importance of addressing these vulnerabilities promptly to fortify the Merkury application's security posture and safeguard user data and privacy.

6.1.5 Dynamic Analysis – Network Analysis

Network traffic analysis of the bulb yielded information about the bulb's hardware and the cloud infrastructure behind the bulb.

I. ESP32 Wireless Hardware

Wireshark output identified the bulb's hardware source as 'Espressi_3d', associated with Espressif embedded wireless chipsets. The chipset revealed by frames in the bulb traffic is the ESP32 wireless SoC, which provides for Wifi and Bluetooth connectivity [5].

The ESP32 exhibits vulnerabilities notably in its hardware AES core, used for flash encryption to prevent firmware cloning. The Flash Encryption feature in ESP32 is susceptible to Side Channel Attacks or Body Biasing Injection. Side Channel Attacks demand physical access to the device to extract encryption keys from the hardware AES core and the Flash Encryption key stored in the eFuse. Body Biasing Injection, also requiring physical access, exploits the temporal locality of AES operation. Consequences of these vulnerabilities include potential firmware tampering, allowing for the introduction of malicious code or backdoors, firmware cloning for analysis and reverse engineering, and the risk of stealing WiFi credentials stored in the firmware or flash memory for automatic reconnection [5]. These security concerns emphasize the importance of addressing and mitigating these vulnerabilities to ensure the robustness of ESP32-based IoT devices like smart light bulbs.

II. Amazon Web Services

When toggling the Merkury bulb through the app, the bulb communicated with 54.68.229.105, an IP address associated with an AWS Elastic Compute instance.

This gives us insight into how the service functions with minimal delays. This instance is in the US-West-2 zone, providing low latency for a round trip request from the research site.

6.2 Cync by GE

6.2.1 Weak encryption

The application allows multiple versions of TLS and SSL though the class has been labeled deprecated, a downgrade attack, followed by any attack which TLS 1.1, 1.2, 1.3, and SSL 3.0 is vulnerable to can be used by an attacker to exploit the application. For example, Poodle attack (CVE-2014-3566).

```

55 # static fields
56 .field private static final synthetic $VALUES:[Ljava/lang/Enum;
57
58 .field public static final Companion:iohttp3/TlsVersion$Companion;
59
60 .field public static final enum SSL_3_0:iohttp3/TlsVersion;
61
62 .field public static final enum TLS_1_0:iohttp3/TlsVersion;
63
64 .field public static final enum TLS_1_1:iohttp3/TlsVersion;
65
66 .field public static final enum TLS_1_2:iohttp3/TlsVersion;
67
68 .field public static final enum TLS_1_3:iohttp3/TlsVersion;
69
70
71 # instance fields
72 .field private final javaName:Ljava/lang/String;
73

```

Figure 2: Deobfuscated code snippet with multiple TLS, and SSL versions

6.2.2 Weak Cryptographic Algorithms

The security analysis of the application has revealed a critical issue regarding its handling of random number generation. The application is currently using a weak fallback random number generator for bluetooth services, which poses significant security risks as bluetooth transactions have sensitive data and can be used to craft an attack due to its vulnerability to known exploits. This vulnerability stems from the fact that the generator can produce predictable results, undermining the randomness required for secure operations.

Specifically, the application utilizes 'java.util.Random' instead of the more secure 'java.security.SecureRandom'. While 'java.util.Random' is suitable for general purposes where randomness is not a critical factor, it falls short in scenarios demanding high security. This is because 'java.util.Random' is not designed to withstand attacks that exploit predictable random number generation. In contrast, 'java.security.SecureRandom' is specifically tailored for security applications, providing a much stronger and less predictable source of randomness.

```

11 .end annotation
12
13 # static fields
14 .field public static final bpdqbp:Ljava/lang/String; = "LoginAction"
15
16 .field public static final pqdbppq:I = 0x22
17
18
19 # instance fields
20 .field public final bpdqbp:[B
21
22 .field public bpdqbp:Lcom/tuya/sdk/bluetooth/bdppppqddqppb;
23
24 .field public bpbppqb:Ljava/util/Random;
25
26 .field public bdddb:Landroid/os/Handler;
27
28 .field public bpdqdp:Lcom/tuya/sdk/blelib/connect/response/BleReadResponse;
29
30 .field public final pdappqb:[B
31
32 .field public pppbpb:Ljava/lang/String;
33
34 .field public final qddqpp:[B
35
36 .field public qppdq:I
37
38
39 # direct methods
40
41 .method public constructor <init>()V
42

```

Figure 3: Snippet of code showing initialization of java.util.random

Secondly, the security analysis of the IO stream reveals the use of MD5 and AES 128 (ECB) for cryptographic operations. However, there are significant concerns regarding the implementation of MD5. Due to inherent weaknesses in the MD5 algorithm, it is possible to generate different inputs that yield identical hash values. This vulnerability poses serious security risks, as it can lead to hash collision attacks. Additionally, our investigation found no evidence of salting being employed in hash computations. There is no salting done, the absence of salting increases the vulnerability of hashes to rainbow table attacks. In such a scenario, hashes become significantly more susceptible to being cracked, compromising the security of the data.

```

32
33 .method public static computeMD5Hash(Ljava/io/InputStream;)J
34 .locals 6
35
36 new-instance v0, Ljava/io/BufferedInputStream;
37
38 invoke-direct {v0, p0}, Ljava/io/BufferedInputStream;.<init>(Ljava/io/InputStream;)V
39
40 :try_start_0
41 const-string p0, "MD5"
42
43 invoke-static {p0}, Ljava/security/MessageDigest;.>getInstance(Ljava/lang/String;)Ljava/security/MessageDigest;
44
45 move-result-object p0
46
47 const/16 v1, 0x4000
48
49 new-array v2, v1, [B
50

```

Figure 4: Weak cryptography for IO stream

6.2.3 Unsafe Library Usage

The examination of the codebase deobfuscated using APKlab and Ghidra has revealed multiple instances where the functions 'memcpy' and 'Memset' are being used. This raises substantial security concerns, particularly in relation to potential buffer overflow attacks. The use of these functions, especially in the absence of adequate safeguards, can render the code unsafe and susceptible to

such attacks. Buffer overflow vulnerabilities occur when data exceeds the allocated buffer memory, leading to the overwriting of adjacent memory. This can result in erratic program behavior, crashes, and, most critically, a doorway for attackers to execute arbitrary code.

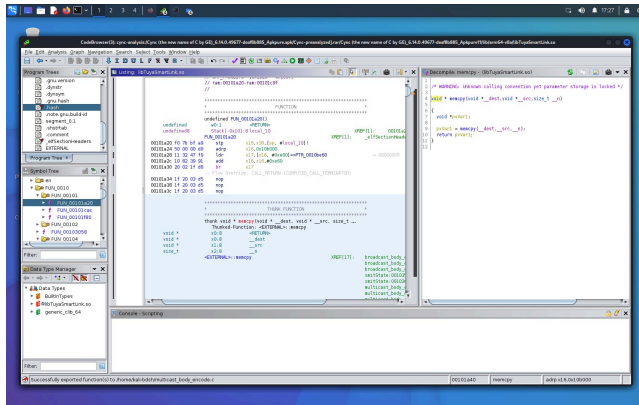


Figure 5: Vulnerable library usage (memcpy)

We also identified a potential integer overflow vulnerability associated with the implementation of the 'memalign' function in the decryption program. In earlier versions of the GNU C Library, there have been documented issues where these functions could return a pointer to a heap area that is smaller than requested. This specific vulnerability, identified as CVE-2018-6485, poses a significant risk as it can lead to heap corruption.

Heap corruption occurs when the allocated memory block is smaller than required, leading to scenarios where data might overflow into adjacent memory blocks. This can corrupt the data in the heap, cause program crashes, and potentially allow attackers to exploit the corrupted heap to execute malicious code. The vulnerability stems from the improper handling of large size values in 'memalign', which could lead to integer overflow. Consequently, this can result in the allocation of a heap area that is inadequate for the intended use, thereby exposing the system to potential security breaches.

To address this vulnerability, it is essential to review the usage of 'memalign' in the decryption program and consider updating or replacing it with safer alternatives.

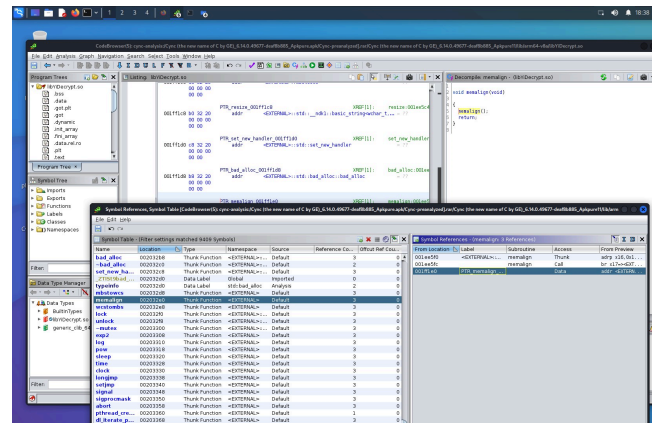


Figure 6: Vulnerable library usage (memalign)

6.3 Philips Hue

6.3.1 Permission Issues

From the MobSF analysis results, our team found a major fallacy in terms of permissions being used. To provide some context, the Philips Hue application has a privacy manual in which they note where and how each permission is to be used:

Usage Data relating to our products and their functionalities	We utilize this data in order to provide the requested functionalities and where applicable to improve or develop our products and new technologies via analytics, for personalization of our product functionalities, and for our marketing efforts.	- Device information, - log information, - location information - to the extent this is strictly necessary for us to provide you with the requested use of our services. This is necessary in our Philips Hue application for instance to allow your lights to be properly triggered through certain routines you choose to set up, such as triggers relating to the time of sunrise and sunset at your location or when you come home or leave your home. Location data provided by your device using these services will remain in your device and in the Philips Hue Bridge.
		With regards to our Philips Hue Bluetooth application, location data is provided through your operating system because this is a requirement when enabling the Bluetooth connection to your lights. Your location data will not be shared with us.
		other information about your use of our products, other information about your setup.

Figure 7: how and where the user location information is going to be used is clearly stated in the privacy policy

This could lead to very serious legal consequences if it is true that the Philips Hue application is using camera permissions of the user device without asking for consent.

Static analysis of the code also revealed interesting findings including the usage of deprecated hashing algorithms.

```

1 (String str2 = md5Hex.string(str3)) {
2     String md5hash = BrowserLogger.getLogger("user_id_hash_value", null);
3     if (md5hash == null || str2.length() == 0) {
4         return getSuffixFromUseridHashAndKey(str2, str3);
5     }
6     BrowserLogger.browseLogDefault(BrowserLogger.INSTANCE, TAG, BrowserLogger.Priority.V, (Throwable) null, (y.f.a) f113484b,
7     BrowserLogger.browseLogDefault(BrowserLogger.INSTANCE, TAG, BrowserLogger.Priority.V, (Throwable) null, (y.f.a) new b(str2, str3), 4,
8     String md5hash = getMd5hash(str3);
9     sharePreferences.edit().putString("user_id_key", str3).putString("user_id_hash_value", md5hash).apply();
10    return getSuffixFromUseridHashAndKey(md5hash, str2);
11 }

```

Additionally, another deprecated hashing algorithm known as SHA-1 is used in a error tracking and performance monitoring library known as Sentry:

```
private static final Charset F221717a = Charset.forName("UTF-8");

public static String a(String str, int i) throws IOException {
    if (str == null || str.isEmpty()) {
        try {
            return new StringBuilder(new BigInteger(1, MessageDigest.getInstance("SHA-1").digest(str.getBytes("F221717a"))).toString(16)).toString();
        } catch (NoSuchAlgorithmException e10) {
            throw new RuntimeException(e10);
        }
    }
    try {
        MessageDigest e4_1b90 = MessageDigest.getInstance("SHA-1");
        e4_1b90.update(str.getBytes("F221717a"));
        try {
            return new String(e4_1b90.digest(), "UTF-8");
        } catch (Throwable th2) {
            throw new RuntimeException("SHA-1 isn't available to calculate the hash.", th2);
        }
    } catch (Throwable th3) {
        throw new RuntimeException("SHA-1 isn't available to calculate its hash.", th3, str);
    }
}

return null;
}
```

Overall, while there is use of deprecated hashing algorithms, both MD5 and SHA-1 might still be adequate for internal identifiers or checksums if the hash represents various elements like user IDs, device IDs, or session IDs, which is only used in logging and monitoring without exposing the actual data.

```
public final String a() {
    if (this.b != null && (this.b.intValue() < 0 || this.b.intValue() > 254)) {
        return "brightness should be a value between 0 and 254";
    }
}
```

Another major hurdle was our collective lack of prior experience in hacking mobile apps and understanding mobile security systems. Engaging with real-life products adhering to rigorous security standards proved more daunting than anticipated, requiring us to rapidly learn and

apply complex concepts in a practical, high-stakes environment. These challenges, while difficult, provided us with invaluable learning experiences.

ACKNOWLEDGMENTS

We would like to express our heartfelt gratitude to Professor Patrick Tague for his invaluable guidance and support throughout our project. His dedication in setting up this collaborative venture was instrumental in fostering a dynamic and productive working environment. His insights and leadership significantly enriched our learning experience and underpinned our team's success.

We are also immensely grateful to our fellow students for their constructive feedback at every checkpoint. Their thoughtful critiques and suggestions were crucial in the continuous refinement of our project. Their perspectives encouraged us to embrace diverse approaches and facilitated a collaborative learning environment. Their support and engagement have not only enhanced our project but also strengthened our sense of community and collective growth. We deeply appreciate their contributions and camaraderie, which have been pivotal to our professional development.